1. Langage

J'ai fait le projet en Python, parce que le code est plus lisible qu'en C et que je le maîtrise mieux.

2. Notation utilisée

Pour décrire le coût des algorithmes, j'utilise la notation O. Elle ne donne pas un temps en secondes, elle indique comment le temps de calcul ou la mémoire augmente quand le nombre de colis augmente. On note n le nombre de colis. O(1) veut dire que le coût ne change pas, qu'il y ait 100 ou 100 000 colis. O(n) veut dire que le coût grandit comme le nombre de colis : avec 10 fois plus de colis, c'est 10 fois plus long. $O(n^2)$ veut dire qu'avec 10 fois plus de colis, c'est 100 fois plus long. O(n log n) se situe entre les deux : avec 10 fois plus de colis, c'est un peu plus de 10 fois plus long. Enfin, O(n × C) veut dire que le coût dépend à la fois du nombre de colis et de la capacité C du camion.

3. Stockage et recherche par identifiant

Le programme doit retrouver rapidement un colis à partir de son identifiant, même avec 100 000 colis. Je stocke les colis dans une liste et dans un dictionnaire qui associe chaque identifiant à son colis. La recherche est en O(1), c'est à dire qu'elle prend le même temps quel que soit le nombre de colis, car le dictionnaire trouve directement l'identifiant. L'ajout est aussi en O(1). En mémoire, c'est du O(n). L'alternative était de parcourir la liste à chaque recherche, en O(n), ce qui devient plus long à mesure que le nombre de colis augmente. La recherche étant l'opération la plus fréquente, le dictionnaire est plus adapté.

4. Tri des colis

Il faut pouvoir afficher les colis par poids, par valeur ou par priorité. J'ai utilisé un tri fusion : on coupe la liste en deux, on trie chaque moitié, puis on fusionne les deux moitiés triées. Le temps est en O(n log n) et la mémoire en O(n). L'alternative était le tri par insertion, en $O(n^2)$. Je l'ai écarté parce qu'il devient beaucoup trop lent avec un grand nombre de colis.

5. Chargement du camion

Il faut choisir des colis dont le poids total ne dépasse pas la capacité du camion, en obtenant la plus grande valeur possible. J'ai utilisé la programmation dynamique. Le programme calcule d'abord la meilleure valeur possible pour un camion de 1 kg, puis de 2 kg, et ainsi de suite jusqu'à la vraie capacité. Pour chaque colis, il regarde s'il vaut

mieux le prendre ou non. Il garde aussi ses décisions en mémoire pour retrouver à la fin les colis chargés. Le temps et la mémoire sont en O(n × C).

J'ai choisi cette méthode parce que le sujet demande la meilleure valeur possible, et elle donne toujours la solution optimale. L'alternative était un algorithme glouton, qui prend d'abord les colis qui rapportent le plus par kilo. Il est plus rapide, en O(n log n), mais pas toujours optimal. Avec les colis du sujet et 20 kg de capacité, il obtient 57 € alors que la programmation dynamique obtient 63 €.

6. Prise en compte de la priorité

La priorité va de 1 à 5, 5 étant le plus urgent. Dans le calcul du chargement, je remplace la valeur par un score qui augmente avec la priorité. Un colis de priorité 5 compte comme s'il valait deux fois plus cher, alors qu'un colis de priorité 1 ne gagne que 20 %. J'ai choisi de multiplier la valeur plutôt que d'ajouter des points, pour que la priorité favorise les colis urgents sans qu'un colis presque sans valeur soit chargé uniquement grâce à elle. Le poids reste la limite à respecter.

7. Performances

J'ai testé le programme avec 100, 1 000, 10 000 et 100 000 colis.

Le tri est en O(n log n). Avec 10 fois plus de colis, il prend un peu plus de 10 fois plus de temps, et il trie 100 000 colis en une demi seconde environ. Il reste adapté. On pourrait l'améliorer en gardant des listes déjà triées pour ne pas tout retrier à chaque affichage.

Le sac à dos est en O(n × C). Pour un même camion, avec 10 fois plus de colis, il prend environ 10 fois plus de temps, et il traite 100 000 colis en un tiers de seconde. Il reste adapté pour un vrai camion.